

Revert string support in `std::constant_wrapper`

Document #: P4206R0 [\[Latest\]](#) [\[Status\]](#)
Date: 2026-04-30
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Zach Laine
<whatwasthataddress@gmail.com>
Matthias Kretz
<m.kretz@gsi.de>
Jonathan Wakely
<cxx@kayari.org>
Tomasz Kamiński
<tomaszkam@gmail.com>

Contents

- [1 Introduction](#)
- [2 Usage of `constant\_wrapper`](#)
- [3 String Support](#)
- [4 Future Language Evolution](#)
- [5 Proposal](#)
 - [5.1 Implementation Status](#)
 - [5.2 Design Note](#)
 - [5.3 Wording](#)
 - [5.4 Feature-Test Macro](#)
- [6 References](#)

§ 1 Introduction

Originally, the design of `std::constant_wrapper` [\[P2781R4\]](#) was :

```
template <auto X, class = remove_cvref_t<decltype(X)>>
struct constant_wrapper {
    // operators and stuff
};

template <auto X>
constexpr auto cw = constant_wrapper<X>();
```

In [\[P2781R5\]](#) of the paper, the shape changed significantly in order to support strings, and was eventually adopted in this form:

```
template <typename T>
struct cw-fixed-value; // exposition-only

template <cw-fixed-value X, class = typename decltype(X)::type>
struct constant_wrapper {
    // operators and stuff
};

template <cw-fixed-value X>
constexpr auto cw = constant_wrapper<X>();
```

We think that this change harms the usability of `std::constant_wrapper` for little benefit. The string support isn't much in the way of support, and can be achieved with future language improvements — but if we keep the current shape, the usability is bad forever. We should revert it.

§ 2 Usage of `constant_wrapper`

Generally speaking, `constant_wrapper` has two broad, overlapping uses:

1. just wrapping a constant in a way that's preserved through layers of function calls, such that the eventual receiver can deduce a value and make use of it.
2. as a DSL where the operators preserve constant-ness, e.g. `cw<5> + cw<3>` yields `cw<8>` rather than simply `8`.

This paper is going to focus on the first one.

A fairly typical use of constant wrapping would be:

```
template <int I>
auto f(std::constant_wrapper<I>) -> void {
    // ...
}

auto main() -> int {
    f(std::cw<5>);
}
```

But, as [/u/Massive-Bottle-5394](#) pointed out on [r/cpp](#) soon after the Croydon meeting, this actually *does not work* with `std::constant_wrapper`. The error you get is:

```
<source>:9:6: error: no matching function for call to 'f(const std::constant_wrapper<std::_CwFixedValue<int>
    {5}, int>&)'
  9 |     f(std::cw<5>); // error
    |     ~~~~~
• there is 1 candidate
• candidate 1: 'template<int I> void f(std::constant_wrapper<(std::_CwFixedValue<int>)I>)'
  <source>:4:6:
    4 | auto f(std::constant_wrapper<I>) -> void {
      |     ^
• template argument deduction/substitution failed:
• mismatched types 'int' and 'const std::_CwFixedValue<int>'
  <source>:9:6:
    9 |     f(std::cw<5>); // error
      |     ~~~~~
```

It is fairly surprising that this doesn't work. A pretty typical way of providing a simple constant wrapper since C++17 is:

```
template <auto V>
using constant = integral_constant<decltype(V), V>;
```

For instance, Boost.Mp11's [mp_value](#) is implemented like this. And the above example *does* work with this formulation.

But it doesn't work with C++26's `std::constant_wrapper` because `std::cw<5>` does not give you a `constant_wrapper<int(5)>`, it yields a `constant_wrapper<cw-fixed-value<int>(5)>`. And `cw-fixed-value<int>(5)` is not some kind of `int`, hence deduction fails.

You could instead do this:

```
template <auto I>
auto f(std::constant_wrapper<I>) -> void {
    // ...
}

auto main() -> int {
    f(std::cw<5>); // ok
}
```

But now, what can you do with `I`? It's a value of an exposition-only type. It's not an `int` or convertible to one. The ways to get at that `int` are to go through the wrapper:

```
template <auto I>
auto f(std::constant_wrapper<I> c) -> void {
    constexpr int A = I;           // error
    constexpr int B = std::cw<I>.value; // ok
    constexpr int C = c.value;      // ok
}
```

```
constexpr int D = c;           // ok
}
```

This means that the template parameter itself is somewhat useless, you might as well just use `c`.

Now, the original formulation of `f` also *required* that the constant be an `int`. How do we do that with `std::constant_wrapper`? The easiest way is recognizing that there actually is a second template parameter:

```
template <auto I>
auto f(std::constant_wrapper<I, int>) -> void {
    // ...
}

auto main() -> int {
    f(std::cw<1>); // ok
    f(std::cw<2u>); // error
}
```

Which is also somewhat surprising, since that template parameter isn't really there to be user facing, it's just an implementation detail to get argument-dependent lookup to work.

One consequence of this implementation detail that the name and type of the implementation detail are exposed in compiler diagnostics, but another is increased symbol sizes. For instance:

```
auto f(auto x) -> void;

auto g() -> void {
    f(std::cw<1>);
}
```

The mangled name of the function being invoked here is:

Revision	Mangled	Demangled
R4	<code>_Z1fISt16constant_wrapperILi1EiEEvT_</code>	<code>void f<std::constant_wrapper<1, int> > (std::constant_wrapper<1, int>)</code>
R5	<code>_Z1fISt16constant_wrapperIXtlSt13_CwFixedValueIiELi1EEEEiEEvT_</code>	<code>void f<std::constant_wrapper<std::_CwFixedValue<int> {1}, int> > (std::constant_wrapper<std::_CwFixedValue<int> {1}, int>)</code>

That's a change from 36 characters to 61.

Another issue is the question of address uniqueness. Consider:

```
template <auto V> auto foo() -> void const* { return &V; }
template <auto V> auto bar(std::constant_wrapper<V> cw) -> void const* { return &cw.value; }
```

For an object `o` of class type, `foo<o>()` and `bar(std::cw<o>)` do not return the same address — because the former returns the address of the template parameter object, but the latter returns the address of a subobject of a template parameter object of different type. If your program happens to depend on both formulations returning the same address, this can be a difficult bug to track down.

§ 3 String Support

This raises the question: what benefit do we get from this less convenient interface? The stated motivation for the change was to support strings. What does that support look like?

In the original design, `std::cw<"foo">` was ill-formed because we're not allowed to pass a pointer to a string literal as a constant template argument. Now, it is valid — by virtue of wrapping the string literal in an array. And the other DSL stuff just works too, so `std::cw<"foo">[std::cw<0>]` yields `std::cw<'f'>`. And these are null terminated, so `std::cw<"foo">[std::cw<3>]` yields `std::cw<'\0'>`.

But, outside of being able to write `std::cw<"foo">`, what string operations can you do?

You cannot do comparisons:

```
auto a = std::cw<"foo">;
auto b = std::cw<"bar">;
a == b; // error
```

You cannot use non-literal strings:

```
constexpr char const* msg = "amount";
auto c = std::cw<msg>; // error
```

And on the receiver side, as in the previous section, you can't do anything with the template argument until you wrap it locally into something like a `std::string_view`, after also ensuring that we do get something like a string:

```
template <auto V, size_t N>
auto f(std::constant_wrapper<V, char const[N]>) -> void {
    constexpr auto S = std::string_view(std::cw<V>.value);
}

auto main() -> int {
    f(std::cw<"amount">);
}
```

Now, it is true that string support in C++26 template arguments is still weak. `std::string` doesn't work because we don't have non-transient constexpr allocation, `std::string_view` isn't structural, so users have to write their own fixed-length string literal types — like the `strlit<N>` illustrated in [R4](#).

But such a thing actually gives meaningful string support, with some additional work:

```
template <strlit S>
struct string_constant : constant_wrapper<S> {
    // possibly some string specific operations here
    // like a conversion to string_view.
    // but even if string_constant<S> were an alias
    // to constant_wrapper<S>, that's a big improvement
};

template <strlit S>
inline constexpr auto cstr = string_constant<S>();
```

For example, [here](#) those types are named `fixed_string` and `constexpr_string`.

This requires explicit typing on the user side, but has the added benefit that more string operations work directly:

```
static_assert(cstr<"bob"> == cstr<"bob">); // ok

template <strlit S>
auto f(string_constant<S>) -> void {
    if constexpr (S == "alice") {
        // ...
    } else if constexpr (S == "bob") {
        // ...
    }
}
```

§ 4 Future Language Evolution

We made `std::cw<"eve">` work, at the cost of making the whole type less convenient to use properly, to work around a language limitation. But it's worth considering what we could do in the language to solve this problem instead and how likely those changes are:

- [\[P0424R2\]](#) ([String literals as non-type template parameters](#)) proposed making string literals work as constant template arguments by way of implicitly transforming them into static storage duration arrays. [\[P3380R1\]](#) ([Extending support for class types as non-type template parameters](#)) picked up that idea and proposes the same. This would make `std::cw<"eve">` just work directly.
- Extending support for structural types in general ([\[P3380R1\]](#)) would allow us to make `std::cw<"eve"sv>` also work.
- Solving non-transient allocation would even allow us to make `std::cw<"eve"s>` work.

If we get these language changes, will we regret having made the array change to `std::constant_wrapper`? Absolutely. The lack of string support is temporary, and isn't really solved by the workaround in `std::constant_wrapper` anyway, but the API of `std::constant_wrapper` is permanent.

Moreover, removing the string literal support from `std::constant_wrapper` doesn't even prevent using it with strings in the present day. With a suitable `strlit<N>/fixed_string<N>` type (or a dynamically typed solution using `std::define_static_string`), users can add a `cstr<"alice">` or `cw_str<"bob">` or however they want to spell it, and get all the functionality with minor cost.

§ 5 Proposal

As a DR against C++26: Revert the `std::constant_wrapper` and `std::cw` design to the R4 shape, where both templates just take an `auto` parameter, and remove `std::cw-fixed-value`.

§ 5.1 Implementation Status

Both libstdc++ and libc++ have already implemented the `std::constant_wrapper` design that we shipped in C++26, and will ship a release with that utility. Nevertheless, both implementations support making the change proposed here as a C++26 DR.

§ 5.2 Design Note

The status quo is that we have:

```
template <cw-fixed-value X>
struct constant_wrapper {
    static constexpr const auto& value = X.data;
};
```

In this scenario, `X.data` is always an lvalue, so `constant_wrapper<V>::value` is always a reference to a static storage duration object. Specifically, it is not a reference to a temporary object. However, if we change the design to:

```
template <auto X>
struct constant_wrapper {
    static constexpr const auto& value = X;
};
```

Then for scalar types, `X` is a prvalue, which means that `value` is a reference to lifetime-extended temporary. The consequence of that would be that `constant_wrapper<42>::value` is not usable as a constant template argument for a template parameter of reference type:

```
template <auto X>
struct constant_wrapper {
    static constexpr const auto& value = X;
};

template <int const& R>
auto f() -> int;

int r = f<constant_wrapper<42>::value>(); // error
```

That would be pretty surprising also, so instead we specify it this way:

```
template <auto X>
struct constant_wrapper {
    static constexpr decltype(auto) value = (X);
};
```

When `X` is a class type, `value` is the same lvalue reference to constant object that it was before. But for prvalues, `value` is now simply a value, so everything continues to work.

§ 5.3 Wording

Change 22.2.1 [utility.syn]:

```

namespace std {
    // [const.wrap.class], class template constant_wrapper
-   template<class T>
-   struct cw-fixed-value;           // exposition only

-   template<cw-fixed-value X, class = typename decltype(X)::type>
+   template<auto X, class = decltype(X)>
    struct constant_wrapper;

    template<class T>
        concept constexpr-param =           // exposition only
            requires { typename constant_wrapper<T::value>; };

    struct cw-operators;                 // exposition only

-   template<cw-fixed-value X>
+   template<auto X>
        constexpr auto cw = constant_wrapper<X>{};
}

```

Change 21.3.5 [\[const.wrap.class\]](#).

```

namespace std {
-   template<class T>
-   struct cw-fixed-value {           // exposition only
-       using type = T;               // exposition only
-       constexpr cw-fixed-value(type v) noexcept : data(v) {}
-       T data;                       // exposition only
-   };
--   template<class T, size_t Extent>
-   struct cw-fixed-value<T[Extent]> { // exposition only
-       using type = T[Extent];       // exposition only
-       constexpr cw-fixed-value(T (&arr)[Extent]) noexcept;
-       T data[Extent];              // exposition only
-   };
--   template<class T, size_t Extent>
-   cw-fixed-value(T (&)[Extent]) -> cw-fixed-value<T[Extent]>; // exposition only

    struct cw-operators {             // exposition only
        // ...
    };

-   template<cw-fixed-value X, class>
+   template<auto X, class T>
        struct constant_wrapper : cw-operators {
-       static constexpr const auto & value = X.data;
+       static constexpr decltype(auto) value = (X);
        using type = constant_wrapper;
-       using value_type = decltype(X)::type;
+       using value_type = decltype(X);

        template<constexpr-param R>
            constexpr auto operator=(R) const noexcept
                -> constant_wrapper<value = R::value> { return {}; }

        constexpr operator decltype(value)() const noexcept { return value; }

        template<class... Args>
            static constexpr decltype(auto) operator()(Args&&... args) noexcept(see below);
        template<class... Args>
            static constexpr decltype(auto) operator[](Args&&... args) noexcept(see below);
    };
}

```

Remove the constructor in 21.3.5 [\[const.wrap.class\]](#)/4, and add a requirement that you can't do `constant_wrapper<42, float>()`.

- ¹ The class template `constant_wrapper` aids in metaprogramming by ensuring that the evaluation of expressions comprised entirely of `constant_wrapper` are core constant expressions ([`expr.const.core`]), regardless of the context in which they appear. In particular, this enables use of `constant_wrapper` values that are passed as arguments to `constexpr` functions to be used in constant expressions.
- ² If a specialization of `constant_wrapper` is instantiated with a type `T` such that `is_same_v<T, value_type>` is false, the program is ill-formed. [Note 1: The unnamed second template parameter to `constant_wrapper` is present to aid argument-dependent lookup

([basic.lookup.argdep]) in finding overloads for which `constant_wrapper`'s wrapped value is a suitable argument, but for which the `constant_wrapper` itself is not. — *end note*]

[*Example 1*:

`// ...`

— *end example*]

~~`constexpr cw_fixed_value(T (&arr)[Extent]) noexcept;`~~

~~*Effects:* Initialize elements of `data` with corresponding elements of `arr`.~~

§ 5.4 Feature-Test Macro

Bump the value of `__cpp_lib_constant_wrapper` in 17.3.2 [[version.syn](#)]:

```
- #define __cpp_lib_constant_wrapper 202603L // freestanding, also in <utility>
+ #define __cpp_lib_constant_wrapper 2026XXL // freestanding, also in <utility>
```

§ 6 References

[P0424R2] Louis Dionne, Hana Dusíková. 2017-11-14. String literals as non-type template parameters.

<https://wg21.link/p0424r2>

[P2781R4] Zach Laine, Matthias Kretz. 2024-02-11. `std::constexpr_wrapper`.

<https://wg21.link/p2781r4>

[P2781R5] Zach Laine, Matthias Kretz, Hana Dusíková. 2024-11-15. `std::constexpr_wrapper`.

<https://wg21.link/p2781r5>

[P3380R1] Barry Revzin. 2024-12-17. Extending support for class types as non-type template parameters.

<https://wg21.link/p3380r1>